

Documentation de Déploiement - M-Atici CRM

- **Dépôt GitHub (Code source)** : github.com/mero-60160/mon-projet-cda
- **Application en production** : www.m-atici.fr

1. Introduction

Ce document détaille la stratégie de déploiement mise en place pour l'application M-Atici CRM. L'objectif est de garantir un passage en production fluide, sécurisé et entièrement automatisé via une approche CI/CD (Intégration Continue et Déploiement Continu).

2. Architecture de l'Infrastructure et Serveur

L'application est déployée sur un serveur dédié cloud : - **Hébergeur** : VPS OVHcloud - **Adresse IP** : `51.178.54.131` - **Système d'exploitation** : Ubuntu 25.04 (LTS) - **Utilisateur système** : `ubuntu` - **Répertoire de l'application** : `/home/ubuntu/mon-projet-cda`

L'écosystème applicatif est entièrement conteneurisé et repose sur 3 services majeurs : - **Frontend** : Application React (servie par Caddy en reverse-proxy avec SSL Let's Encrypt automatique). - **Backend** : API REST Node.js / Express. - **Base de données** : PostgreSQL 15 (les schémas et migrations sont gérés avec l'ORM Prisma).

L'orchestration globale est assurée par **Docker Compose**.

3. Configuration Initiale du Serveur VPS

Pour pouvoir accueillir le projet, le serveur VPS a été configuré avec les prérequis suivants :

1. Docker & Docker Compose :

```
sudo apt update
sudo apt install -y docker.io docker-compose-v2
sudo usermod -aG docker ubuntu
```

2. Clonage du Projet : Le dépôt a été cloné dans le répertoire utilisateur :

```
cd /home/ubuntu
git clone https://github.com/mero-60160/mon-projet-cda.git mon-projet-cda
```

3. Fichier d'environnement : Un fichier `/home/ubuntu/mon-projet-cda/.env` a été créé pour héberger les secrets de production (non versionnés sur Git) :

```
DATABASE_URL="postgresql://user:password@db:5432/crm_db"  
JWT_SECRET="votre_secret_jwt_de_production"  
EMAIL_USER="votre_email_de_relance"  
EMAIL_PASS="mot_de_passe_d_application"  
PORT=3000
```

4. Configuration Docker

Un fichier `docker-compose.yml` orchestre les services sur le serveur de production.

Avantages de cette approche : - Isolation stricte des services. - Reproductibilité entre l'environnement de développement et de production. - Gestion centralisée des variables d'environnement.

Extrait de configuration (Backend) :

```
backend:
  build: ./backend
  restart: always
  environment:
    - DATABASE_URL=${DATABASE_URL}
    - JWT_SECRET=${JWT_SECRET}
    - EMAIL_USER=${EMAIL_USER}
    - EMAIL_PASS=${EMAIL_PASS}
  ports:
    - "3000:3000"
  depends_on:
    - db
```

5. Pipeline CI/CD (GitHub Actions)

Pour garantir la stabilité de la production, un pipeline complet de CI/CD a été configuré dans `.github/workflows/main.yml`.

À chaque validation de code (`git push`) sur la branche `main` :

1. Intégration Continue (CI) :

- **Build & Install** : GitHub Actions récupère les sources et installe les dépendances Node.js du frontend et du backend.
- **Génération Prisma** : Le client Prisma est généré.
- **Tests unitaires** : La suite de tests Jest est exécutée. Si un test échoue, le pipeline s'arrête immédiatement pour empêcher le déploiement d'un bug en production.

2. Déploiement Continu (CD) :

- Si les tests passent, une connexion SSH sécurisée est ouverte vers le VPS OVH (via la clé privée stockée dans les secrets GitHub).
- Le script de déploiement automatique `deploy-staging.sh` est lancé sur le VPS.

Secrets GitHub configurés sur le dépôt :

- `SSH_HOST` : L'adresse IP du VPS (`51.178.54.131`).
- `SSH_USER` : L'utilisateur SSH (`ubuntu`).
- `SSH_PRIVATE_KEY` : La clé privée SSH autorisée sur le serveur VPS.

(Le fichier de configuration se trouve dans `.github/workflows/main.yml`).

6. Script de Déploiement

Le déploiement proprement dit s'effectue via un script bash (`deploy-staging.sh`) exécuté sur le serveur cible. Ce script automatise les tâches d'exploitation courantes :

1. Mise à jour des sources via Git.
2. Reconstruction des images Docker (`docker compose build`).
3. Démarrage des conteneurs (`docker compose up -d`).
4. Exécution des migrations de base de données en production via Prisma (`npx prisma migrate deploy`).

7. Sécurité et Bonnes Pratiques

- **Secrets** : Les variables sensibles (Mots de passe, clés JWT) ne sont pas versionnées sur Git. Elles sont injectées directement sur le serveur via le fichier `.env` .
- **Certificats SSL** : Le trafic entrant en production est chiffré (HTTPS), géré par Caddy en reverse-proxy qui renouvelle automatiquement les certificats Let's Encrypt.
- **Persistence** : Les données de PostgreSQL sont attachées à des Volumes Docker persistants pour prévenir toute perte de données lors du redémarrage des conteneurs.

8. Tâches planifiées (Cron)

Afin d'automatiser les relances de factures et la sauvegarde de la base de données PostgreSQL, deux tâches `cron` doivent être configurées sur le système hôte du VPS.

Exécutez `crontab -e` en tant qu'utilisateur `ubuntu` et ajoutez les lignes suivantes :

```
# 1. Sauvegarde automatique quotidienne de la base de données (toutes les nuits à 2h00)
0 2 * * * /home/ubuntu/mon-projet-cda/scripts/backup-db.sh > /dev/null 2>&1

# 2. Relance automatique des factures en retard de paiement (tous les matins à 8h30)
30 8 * * * docker compose -f /home/ubuntu/mon-projet-cda/docker-compose.yml exec -T backend node
src/scripts/alerteRetard.js > /dev/null 2>&1
```

(Note : Assurez-vous que le script de sauvegarde est exécutable en lançant `chmod +x scripts/backup-db.sh`).